
Architectuurhandboek: Autonome AI-Hulpservice

Blauwdruk voor een Container-Gestuurde, Data-Driven Routing Infrastructuur

Status: Technisch Ontwerphandboek

Doelstelling: Naadloze WhatsApp & Signal Integratie

Inhoudsopgave

1. De Infrastructuur: Linux Containers

1.1 Hostomgeving en Systeemstabiliteit

1.2 Basis Host Container-Implementatie

1.3 Geïsoleerde Signal REST API Container

2. De Datagestuurde Backend-Architectuur

2.1 Centrale Relatiedatabase (SQLite)

2.2 Poortwachter en Inkomende Webhooks (server.py)

2.3 De Berichtensorteerder en Verwerker (dispatcher.py)

2.4 Het Conceptuele AI-Brein (nurse.py)

3. Functionele Uitbreiding naar Signal-Integratie

1. De Infrastructuur: Linux Containers

1.1 Hostomgeving en Systeemstabiliteit

Voor een betrouwbare, continu beschikbare AI-hulp-service is de stabiliteit van de onderliggende hostomgeving kritiek. In tegenstelling tot mobiele platforms (zoals Android via Termux), die onderhevig zijn aan agressieve energiebesparende mechanismen zoals Doze Mode en de Low Memory Killer, biedt een dedicated laptop of mini-PC een stabiliteit waarmee runtime processen beschermd blijven. Door het systeem in te richten binnen Linux-containers (Docker of LXD), profiteert de architectuur van hardwarematige persistentie en proces-isolatie via kernel namespaces en cgroups.

Dit garandeert dat bij eventuele stroom- of netwerkuitval de status van de wachtrijen en databestanden onbeschadigd blijft en services automatisch herstarten bij een systeem-reboot.

1.2 Basis Host Container-Implementatie

Afhankelijk van de voorkeur voor een systeemcontainer (LXD) of een applicatiecontainer (Docker), kan de basisomgeving met de volgende configuraties worden opgezet.

Optie A: Systeemcontainer via LXD

```
# Initialiseer en start een schone Debian 12 container genaamd 'ai-service'
lxc launch images:debian/12 ai-service

# Voer systeemupdates uit en installeer de vereiste dependencies binnen de container
lxc exec ai-service -- apt update
lxc exec ai-service -- apt install -y python3 python3-pip python3-venv sqlite3 curl
docker.io
```

Optie B: Applicatiecontainer via Docker (Dockerfile)

```
# Dockerfile voor de hoofdapplicatie-runtime
FROM debian:12-slim

RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    python3-venv \
    sqlite3 \
    curl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . /app

RUN python3 -m venv venv && ./venv/bin/pip install fastapi uvicorn requests

CMD ["../venv/bin/python", "server.py"]
```

1.3 Geïsoleerde Signal REST API Container

Om te communiceren met het Signal-netwerk zonder zware bibliotheken in de hoofdcode te hoeven compileren, wordt er een sidecar-container ingericht. Deze maakt gebruik van de voorgebouwde Signal CLI REST API. Deze container draait lokaal en stelt een REST-interface beschikbaar op poort 8080.

```
# Start de Signal CLI REST API service als achtergrondproces
docker run -d \
  --name signal-api \
  --restart always \
  -p 8080:8080 \
  -v ~/.local/share/signal-cli:/home/.local/share/signal-cli \
  bbernhard/signal-cli-rest-api:latest

# Registreer en verifieer het telefoonnummer bij de Signal-dienst
# Vervang +32XXXXXXXXX door het daadwerkelijke nummer van de simkaart
curl -X POST -H "Content-Type: application/json" \
  -d '{"captcha": ""}' \
  http://localhost:8080/v1/register/+32XXXXXXXXX
```

2. De Datagestuurde Backend-Architectuur

2.1 Centrale Relatiedatabase (SQLite)

Het fundament van de Data-Driven Routing-architectuur is een lokale SQLite-database. Elk binnenkomend bericht wordt onmiddellijk gepersisteerd in een centrale tabel. Dit ontkoppelt het netwerk-ontvangstproces volledig van de verwerking en de communicatie met het taalmodel (LLM). De tabel bevat een expliciete kolom `channel` om de herkomst en bestemming van het berichtenverkeer te routeren.

```
# init_db.py
import sqlite3

def init_db():
    conn = sqlite3.connect("messages.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS message_queue (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            sender TEXT NOT NULL,
            message TEXT NOT NULL,
            channel TEXT NOT NULL,
            status TEXT DEFAULT 'pending',
            reply TEXT,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()
    print("Database succesvol geïntialiseerd.")

if __name__ == "__main__":
    init_db()
```

2.2 Poortwachter en Inkomende Webhooks (server.py)

Het script `server.py` fungeert als een asynchrone webserver die poorten openstelt voor inkomende HTTP POST-verzoeken (webhooks). Gebouwd op het FastAPI-framework, neemt dit component berichten aan van zowel Twilio (WhatsApp) als de lokale Signal-API webhook. De taak van dit script is puur transactioneel: het valideert de payload en schrijft deze direct weg naar de database-wachtrij zonder te wachten op de AI-generatie.

```
# server.py
from fastapi import FastAPI, Request
import sqlite3

app = FastAPI()

def queue_message(sender, message, channel):
    conn = sqlite3.connect("messages.db")
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO message_queue (sender, message, channel) VALUES (?, ?, ?)",
        (sender, message, channel)
    )
    conn.commit()
    conn.close()

@app.post("/whatsapp-webhook")
async def whatsapp_webhook(request: Request):
    form_data = await request.form()
    sender = form_data.get("From")
    message = form_data.get("Body")
    if sender and message:
        queue_message(sender, message, "whatsapp")
    return {"status": "queued"}

@app.post("/signal-webhook")
async def signal_webhook(request: Request):
    json_data = await request.json()
    envelope = json_data.get("envelope", {})
    data_message = envelope.get("dataMessage", {})
    sender = envelope.get("source")
    message = data_message.get("message")
    if sender and message:
        queue_message(sender, message, "signal")
    return {"status": "queued"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=5000)
```

2.3 De Berichtensorteerder en Verwerker (dispatcher.py)

De `dispatcher.py` component voert de feitelijke regie over de informatiestroom. Dit proces draait autonoom op de achtergrond, controleert de database op openstaande taken (`pending`), wijzigt de status naar `processing` om race-conditions te voorkomen, consulteert het AI-brein, en distribueert het resultaat op basis van de opgeslagen kanaal-identificator.


```

# dispatcher.py
import sqlite3
import time
import requests
from nurse import generate_nurse_reply

SIGNAL_API_URL = "http://localhost:8080/v1/send"
TWILIO_API_URL = "https://api.twilio.com/2010-04-01/Accounts/YOUR_ACCOUNT_SID/
Messages.json"
TWILIO_AUTH = ("YOUR_ACCOUNT_SID", "YOUR_AUTH_TOKEN")

def send_whatsapp(to_number, text):
    payload = {
        "From": "whatsapp:+14155238886",
        "To": to_number,
        "Body": text
    }
    response = requests.post(TWILIO_API_URL, data=payload, auth=TWILIO_AUTH)
    return response.status_code == 201

def send_signal(to_number, text):
    payload = {
        "message": text,
        "number": "+32XXXXXXXXX",
        "recipients": [to_number]
    }
    response = requests.post(SIGNAL_API_URL, json=payload)
    return response.status_code == 201

def process_queue():
    conn = sqlite3.connect("messages.db")
    cursor = conn.cursor()
    cursor.execute("SELECT id, sender, message, channel FROM message_queue WHERE
status='pending' ORDER BY id ASC LIMIT 1")
    row = cursor.fetchone()
    if row:
        msg_id, sender, message, channel = row
        cursor.execute("UPDATE message_queue SET status='processing' WHERE id=?",
(msg_id,))
        conn.commit()
        ai_reply = generate_nurse_reply(message)
        success = False
        if channel == "whatsapp":
            success = send_whatsapp(sender, ai_reply)
        elif channel == "signal":
            success = send_signal(sender, ai_reply)
        if success:
            cursor.execute("UPDATE message_queue SET status='completed', reply=? WHERE
id=?", (ai_reply, msg_id))
        else:
            cursor.execute("UPDATE message_queue SET status='pending' WHERE id=?",
(msg_id,))
        conn.commit()

```

```
conn.close()

if __name__ == "__main__":
    while True:
        process_queue()
        time.sleep(1)
```

2.4 Het Conceptuele AI-Brein (nurse.py)

De logische encapsulatie zorgt ervoor dat de AI-verpleger (nurse.py) geen weet heeft van het transportmedium (Signal of WhatsApp). Dit component beheert de systeem-prompts en stuurt de gestructureerde JSON-data naar de gekozen inferentie-engine (OpenAI API of een lokale LLM-instantie).

```
# nurse.py
import requests

LLM_API_URL = "https://api.openai.com/v1/chat/completions"
LLM_API_KEY = "YOUR_OPENAI_API_KEY"

SYSTEM_PROMPT = """
Je bent een empathische, zeer deskundige digitale verpleger.
Je luistert aandachtig naar medische of zorggerelateerde vragen van gebruikers.
Geef betrouwbare, rustige en duidelijke informatie.
Wijs de gebruiker er bij ernstige klachten altijd op om contact op te nemen met een arts
of spoedeisende hulp.
"""

def generate_nurse_reply(user_message):
    headers = {
        "Authorization": f"Bearer {LLM_API_KEY}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "gpt-4o",
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_message}
        ]
    }
    try:
        response = requests.post(LLM_API_URL, json=payload, headers=headers)
        if response.status_code == 200:
            return response.json()["choices"][0]["message"]["content"]
            return "Excuses, de medische database is momenteel onbereikbaar."
    except Exception as e:
        return f"Systeemfout bij verwerking zorgvraag: {str(e)}"
```

3. Functionele Uitbreiding naar Signal-Integratie

De modulaire opzet van deze op data gebaseerde routing-architectuur bewijst zijn waarde bij uitbreidingen. Omdat het AI-brein volledig geïsoleerd opereert, vereist de integratie van het Signal-netwerk geen aanpassingen in de kernlogica.

Door de SQLite-wachtrij uit te rusten met de `channel-descriptor` en een parallelle FastAPI-route in `server.py` op te nemen, worden Signal-berichten identiek behandeld als WhatsApp-verzoeken. De `dispatcher.py` minimaliseert platformafhankelijkheden door pas in de allerlaatste stap te differentiëren: hij stuurt uitgaande payloads naar de Twilio-gateway óf naar de lokale Docker-container van de Signal REST API.